SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 30**

Question Count: 5

Code of Conduct

I **R.Pranathi(192525076)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: R. Pranathi

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.

2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.

3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption,

and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
ANS:

The given platform collects billions of website clicks, mobile interactions and customer transactions every day. From this description, we can infer that it needs a distributed big data architecture because a single server cannot store and process such a huge amount of data.

First, the platform would probably use a distributed storage system such as HDFS or cloud data lake storage. The data would be divided and stored across multiple machines. This provides large storage capacity and also protects the system if one machine fails. Historical data can be stored in a data lake for later analysis.

For collecting the data, an ingestion framework such as Apache Kafka would likely be used. Website clicks, mobile activities and transactions are continuously generated, so Kafka can receive and distribute these events to different processing systems. Batch data can also be collected from databases and other sources.

The data would probably be separated based on its type. Structured data, such as customer transactions and sales records, can be stored in tables with fixed columns. Semi-structured data, such as website click logs and JSON data from mobile applications, can be stored in a data lake in its original format. Metadata and suitable indexes can be maintained to make searching and analysis faster.

For batch processing, Apache Spark or Hadoop MapReduce could be used. Spark can process large historical datasets and generate reports such as total sales, customer behaviour and monthly website activity.

For real-time analytics, Kafka with Spark Streaming could be used. When a customer clicks a product or makes a transaction, the event can be processed almost immediately. This helps the company monitor current customer activity and generate real-time reports.

To handle billions of records, the system would use partitioning. Data can be divided based on date, region, customer ID or transaction type. For example, website data can be partitioned by day or month. This allows the processing system to work only on the required part of the data instead of scanning everything.

The platform would also use multiple processing nodes. When the amount of data increases, more machines can be added. This is called horizontal scalability. Replication can also be used so that copies of important data are available if a server fails.

After processing, the analytics results can be stored in a suitable database or data warehouse. Business Intelligence tools can then connect to this analytics layer and display the results through dashboards.

A simple architecture can be shown as:

For example, when thousands of customers click products during an online sale, Kafka can collect the click events, Spark Streaming can process them in real time, and the dashboard can show the most viewed products.

Thus, the probable design uses distributed storage, Kafka for ingestion, Spark/Hadoop for batch processing, Spark Streaming for real-time processing, partitioning for scalability and BI dashboards for reporting. This architecture allows the platform to handle massive workloads while providing both historical and real-time customer analytics.

2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
ANS:

The given e-commerce platform collects different types of customer activities such as product views, cart additions, abandoned checkouts and payment behaviour. Since this information is generated continuously from different regions, the platform would probably use a **distributed and real-time big data architecture**.

First, customer activities can be collected as events using an event streaming system such as **Apache Kafka**. Each activity, such as viewing a product or adding it to the cart, can be sent as an event. Kafka can handle a large number of events from different regions.

The platform would probably maintain customer profiles in a database. Customer details, previous purchases, preferences and behaviour can be stored in a distributed database. Session information can also be maintained so that the system knows what a customer is currently doing.

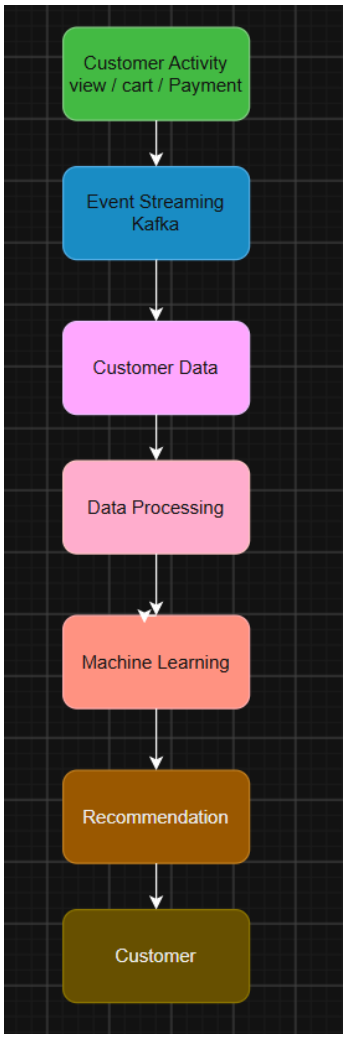
For example, if a customer views a mobile phone, adds it to the cart and then leaves without making payment, these activities can be connected using the customer's ID and session ID. The system can then understand that the customer has shown interest in that product.

The clickstream data and transaction data can be synchronized using common identifiers such as **customer ID, session ID, product ID and transaction ID**. Structured order data can be stored in tables, while browsing information may be stored as semi-structured JSON or log data. These datasets can later be combined for analytics.

A caching system such as **Redis** could be used to store frequently accessed information such as product details, customer recommendations and current sessions. Caching reduces response time because the system does not have to query the main database every time.

For large-scale queries, a distributed query engine such as **Presto/Trino or Spark SQL** could be used. It can combine information from different data sources and produce analytics results.

Machine learning can be used for recommendations. The workflow may be:



The recommendation model can study previous purchases, product views, cart behaviour and similar customers. For example, if a customer repeatedly views laptops, the system may recommend laptops, laptop bags or other accessories.

The platform would also need **scalability and fault tolerance**. Data can be divided into partitions based on customer, region or time. Multiple servers can process different partitions at the same time. Data replication can be used so that another copy is available if one server fails. More servers can also be added when the number of customers increases.

Dashboards and KPI monitoring can be connected to the analytics layer. They can display important measures such as **number of product views, cart abandonment rate, completed orders, conversion rate, sales and payment success rate**.

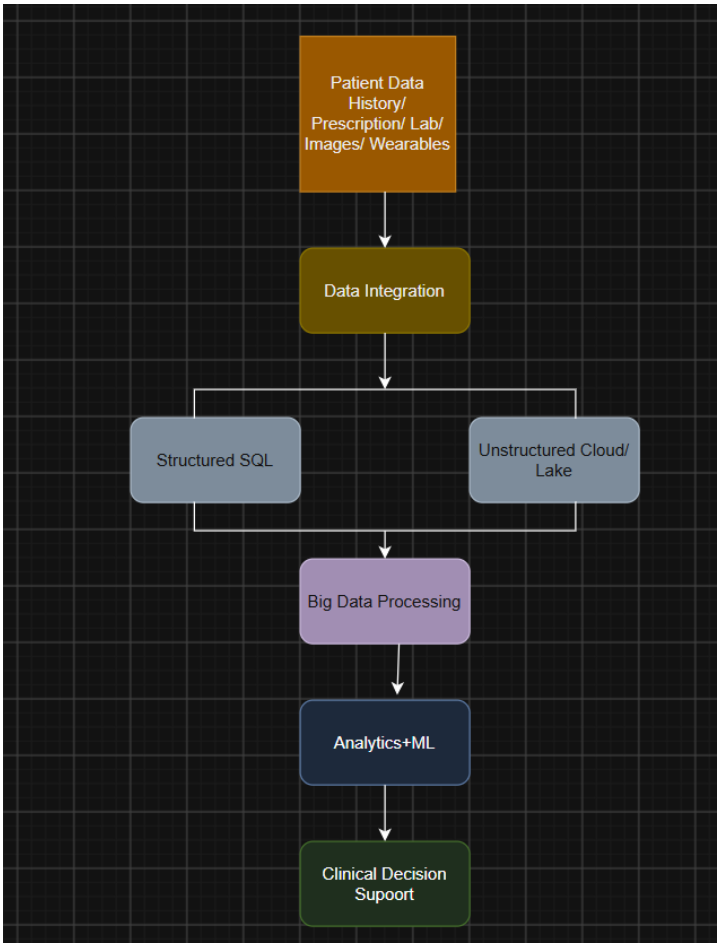
For example, if the dashboard shows that many customers are adding a particular product to their carts but not completing payment, the company can investigate the checkout process or provide a suitable offer.

Therefore, the likely solution combines Kafka for event streaming, distributed databases for customer profiles, Redis for caching, Spark/Trino for distributed queries, machine learning for recommendations and BI dashboards for KPI monitoring. This allows the e-commerce platform to provide real-time personalization while handling large amounts of customer data across different regions.

3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.
ANS:

The given healthcare system handles different types of patient information such as patient history, diagnostic images, wearable sensor data and prescription records. Since these are different types of data, the system would probably use a **hybrid database architecture**. Structured information such as patient details, prescriptions and laboratory results can be stored in relational databases, while diagnostic images and other large files can be stored in distributed or cloud storage.

A possible architecture is:



For integration between hospitals, laboratories and insurance systems, tools such as **APIs, ETL tools and healthcare data standards** would likely be used. These allow information from different systems to be combined. For example, laboratory results from one hospital can be connected with the patient's medical history from another system.

Wearable sensors continuously generate information such as heart rate, activity level and other health measurements. Since this data arrives continuously, a streaming framework such as **Apache Kafka** could be used to collect the sensor data. A processing system such as **Apache Spark Streaming** could then analyze the data in near real time.

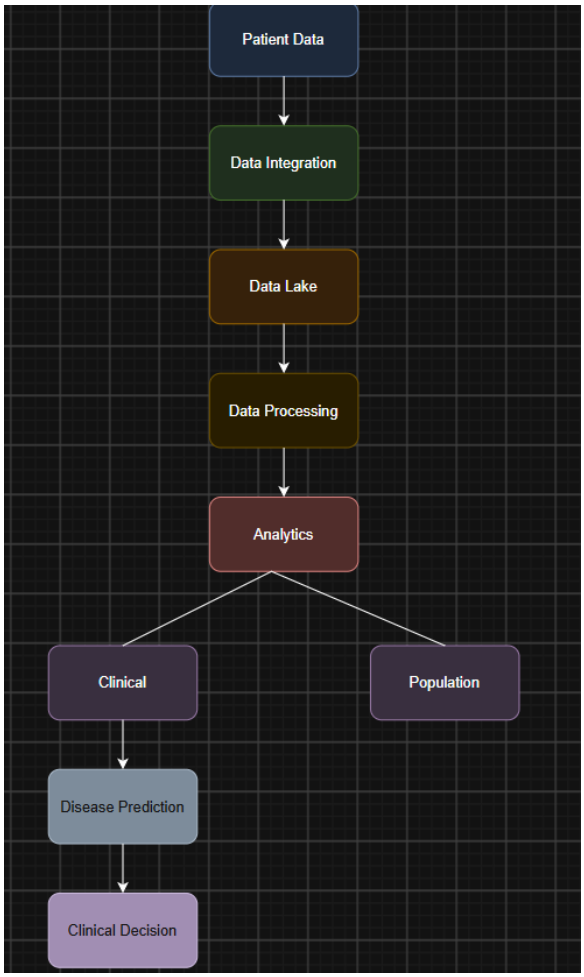
Machine learning can be used for disease prediction and patient monitoring. Historical patient records can be used to train models. The model can identify patterns that may indicate a health risk. For example, abnormal sensor readings combined with patient history may indicate that a patient needs further medical attention.

Privacy is very important in healthcare because patient information is sensitive. Access should be given only to authorized doctors, nurses, laboratories and other approved users. **Role-Based Access Control (RBAC)** can be used for this purpose.

Encryption would probably be used both when data is stored and when it is transferred between systems. Authentication can verify the identity of users, while audit logs can record who accessed or changed patient information.

The platform would also need to follow applicable healthcare privacy and security regulations. Compliance requirements should be included in data storage, access control, sharing and auditing procedures. The exact regulations would depend on the country and healthcare organization.

The analytics pipeline could be:



For example, doctors can use patient history, laboratory results and wearable sensor information together to identify possible health risks. At a larger level, population health analytics can identify disease trends across different regions or age groups.

Thus, the probable healthcare platform would combine **relational databases for structured data, cloud or distributed storage for images and other large data, APIs/ETL for integration, Kafka and Spark for streaming, and machine learning for prediction**. Encryption, access control and auditing would protect patient information, while the analytics pipeline would support clinical decisions and population health studies.

4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.
ANS:

A smart city platform continuously collects data from traffic sensors, CCTV systems, weather stations and public transport systems. Since this data is generated continuously and comes from many locations, the platform would probably use edge computing, distributed messaging and centralized cloud storage.

First, edge computing would likely be used near traffic signals, CCTV cameras and sensors. The edge devices can collect and filter data locally before sending it to the central platform. This reduces network traffic and also allows quick decisions near the source. For example, a traffic sensor can detect heavy traffic and send only important information to the central system.

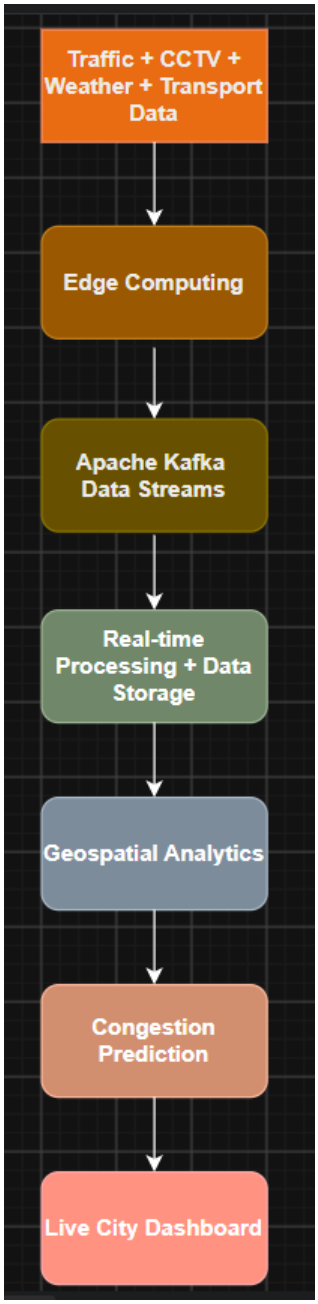
A distributed messaging system such as Apache Kafka could be used to collect continuous data from different city systems. Traffic information, weather updates and transport activity can be sent as separate streams. Kafka can handle a large number of messages and distribute them to different processing systems.

The processed data would probably be stored in centralized cloud or distributed storage. Historical traffic data, weather records and transport information can be stored for future analysis.

For traffic congestion prediction, stream processing can analyze incoming sensor data in near real time. Geospatial analytics can combine vehicle locations, road information and traffic conditions to identify congested areas. A map-based system can then show the affected locations.

Time-series databases would be suitable for sensor data because traffic and weather values change continuously over time. A database such as InfluxDB could store time-based sensor readings. A location-aware database such as PostGIS could be used for geographical information such as roads, vehicle locations and traffic zones.

The basic architecture can be shown as:



Both **real-time and batch analytics** would probably be used. Real-time analytics can identify current traffic congestion and help traffic authorities respond quickly. Batch analytics can study historical traffic patterns and help city planners decide where new roads, parking areas or public transport facilities are required.

For example, historical traffic data may show that a particular road becomes highly congested every evening. City planners can use this information to improve traffic signals or plan alternative routes.

Security is important because the platform contains citizen and infrastructure information. **Authentication and role-based access control** can ensure that only authorized users can access specific data. Encryption can protect information while it is stored and transferred. Audit logs can record important activities and help identify unauthorized access.

Visualization systems would probably connect to the analytics layer and display information through live dashboards. Traffic officers could see congestion areas, road conditions, public transport locations and weather conditions on maps and charts.

For example, a traffic control center can receive a real-time alert when congestion increases at a particular junction. Officials can then change traffic signals or suggest alternative routes.

Thus, the probable smart city platform combines edge computing, Kafka, centralized storage, streaming analytics, geospatial databases, time-series databases, batch processing and live dashboards. This allows city authorities to monitor current conditions and also use historical data for better urban planning.

5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

ANS:

A healthcare analytics system collects different types of patient information such as patient records, wearable device readings, laboratory reports and appointment details. Since this data comes from different sources and contains both structured and unstructured information, a hybrid storage architecture would probably be used.

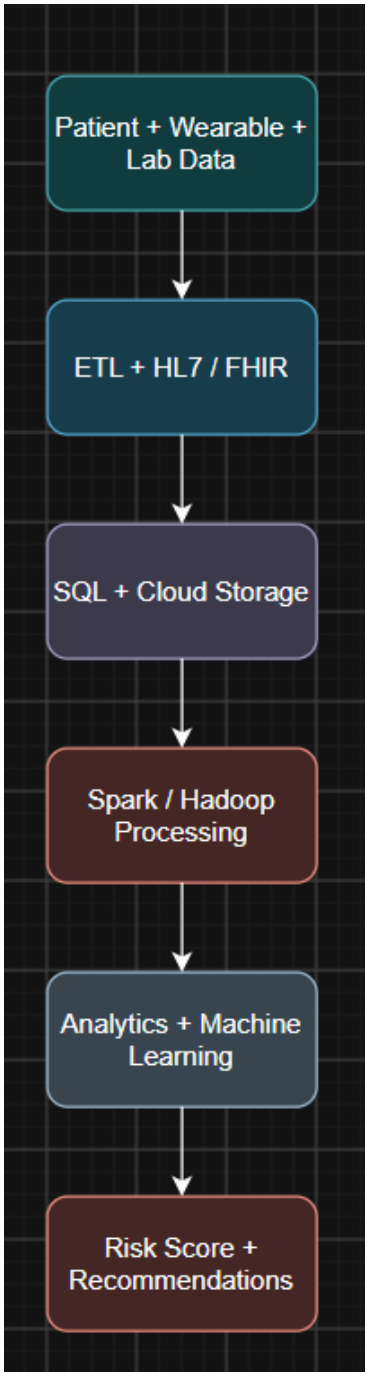
Structured data such as patient ID, appointment details, prescription records and lab values can be stored in a relational database. Unstructured or large data such as medical reports and device data can be stored in a cloud storage or data lake. This allows the system to store large amounts of healthcare data.

The data from hospitals and clinics would probably be connected using ETL pipelines. ETL means Extract, Transform and Load. Data is extracted from different hospital systems, cleaned and converted into a common format, and then loaded into the central healthcare platform.

Healthcare interoperability standards such as HL7 and FHIR could be used to allow different hospital and clinic systems to exchange patient information. This helps combine data even when different organizations use different software systems.

For real-time monitoring, wearable devices can continuously send readings such as heart rate, activity and other health measurements. A streaming system such as Apache Kafka could collect this data, while Spark Streaming could process it in near real time.

The basic architecture can be shown as:



For population-level analysis, distributed processing frameworks such as **Apache Spark or Hadoop** could be used. Healthcare organizations may have data from thousands or millions of patients. Distributed processing divides the work across multiple machines, making large-scale analysis faster.

For example, the system can analyze patient data from different regions to identify the number of people at risk of a particular disease. It can also identify common patterns between age, lifestyle and health conditions.

Machine learning can support **diagnosis risk scoring** by analyzing patient history, lab results and wearable readings. A model can provide a risk score that helps doctors identify patients who may need further examination.

Machine learning can also support treatment recommendations by studying previous patient cases and identifying treatments that were effective for similar conditions. These recommendations should support doctors rather than completely replace medical decisions.

Security and data governance are very important because medical information is sensitive. **Encryption** can protect data while it is stored and while it is transferred. Authentication and **role-based access control** can ensure that only authorized doctors, staff and administrators can access specific information.

The system would also need to follow applicable healthcare privacy and security regulations. **Audit logs** can record who accessed, changed or shared patient information. Regular audits can help identify unauthorized access and maintain accountability.

Therefore, the probable system combines hybrid storage, ETL pipelines, HL7/FHIR interoperability, Kafka and Spark for real-time processing, distributed processing for population analytics, and machine learning for risk scoring and recommendations. Encryption, access control and auditing help protect sensitive patient information and support proper data governance.